

كلية الهندسة المعلوماتية - جامعة دمشق

مشروع مادة المترجمات

لغة Python Flask & Jinja2

إعداد وتقديم الطلاب:

- عمر جمال كحيل
- سيف الدين خالد كناني
- عبد الرحمن مازن السيد احمد
- مهاب تيسير شهاب

أولاً: التعريف بلغة ومفهوم المشروع

في هذا المشروع، قمنا ببناء مترجم (Compiler) متكامل يجمع بين بيئتين أساسيتين في عالم تطوير الويب الحديث:

1. **لغة Python (إطار عمل Flask):** لكتابة منطق النهاية الخلفية (Backend) وتعريف المتغيرات ومسارات الصفحات (Routes).
 2. **محرك القوالب Jinja2:** لبناء واجهات المستخدم (Frontend) وربطها مع البيانات القادمة من بايثون.
- الهدف الأساسي من المشروع هو محاكاة عمل إطار Flask الحقيقي، حيث يمر الكود بعدة مراحل:

1. تحليل كود Python وبناء شجرته النحوية.
2. تحليل قوالب Jinja2 وبناء شجرتها النحوية.
3. استخراج البيانات الوهمية (Mock Data) والتحقق من صحة الكود دلاليًا (Semantic Analysis).
4. ترجمة كل هذا وتوليد كود HTML نقي وربطه بالبيانات لعرضه للمستخدم النهائي.

ثانياً: بناء قواعد اللغة اللفظية والنحوية (Lexical & Syntax Rules)

في البداية، قمنا بكتابة قواعد اللغة اللفظية والنحوية باستخدام لغة قواعد g4. ولأن مشروعنا يتعامل مع لغتين مختلفتين تماماً (بايثون وجينجا)، فقد تطلب الأمر بناء محللين منفصلين، مما أدى إلى إنشاء أربعة ملفات g4:

1. تحليل لغة Python

تم إنشاء ملفي g4 FlaskPythonLexer و g4 FlaskPythonParser. حيث تم تجميع رموز لغة بايثون في الـ Lexer مثل الكلمات المفتاحية (def, return, if, for) والعمليات الحسابية، ثم تم إنشاء القواعد في الـ Parser.

مثال من قواعد ال Parser لبايثون:

```
// تعريف الدوال
functionDecl
decorator* DEF ID LPAREN paramList? RPAREN COLON suite :
    #functionDeclaration
;

// الشروط
ifStatement
IF expression COLON suite :
*(ELIF expression COLON suite)
?(ELSE COLON suite)
ifStatementRule#
;
```

2. تحليل لغة Jinja2 و HTML و CSS

لغة جينجا معقدة لأنها تدمج بين HTML و CSS وكود بايثون الداخلي. لذا قمنا في ملف FlaskJinjaLexer.g4 باستخدام ميزة **الأنماط (Lexical Modes)** في أداة ANTLR للتبديل بين سياقات اللغة المختلفة بسلاسة.

مثال على استخدام ال Modes في Jinja Lexer:

```
// الانتقال من HTML إلى كود Jinja
;TEMPLATE_JINJA_BLOCK_START: '{%' -> pushMode(JINJA_BLOCK_MODE)
;TEMPLATE_JINJA_EXPR_START: '{{' -> pushMode(JINJA_EXPR_MODE)

// الانتقال إلى تنسيقات CSS
;CSS_START: '<style' -> pushMode(CSS_TAG_MODE)
```

بمجرد دخول المترجم في JINJA_BLOCK_MODE، فإنه يبدأ بالتقاط الكلمات المفتاحية الخاصة بجينجا مثل (for, if, extends, block)، وعندما يواجه % { يعود أدراجه إلى النمط الأساسي لمعالجة HTML.

توليد المترجمات بواسطة ANTLR

بعد بناء هذه الملفات، قمنا باستخدام أداة ANTLR4 لتوليد ملفات الجافا الأساسية (ال Lexer وال Parser) لكلتا اللغتين. هذه الملفات شكلت حجر الأساس الذي سيتيح لنا تحويل النصوص البرمجية المجردة إلى أشجار إعراب (Parse Trees) يمكننا التجول داخلها لاحقاً وبناء شجرة النحو المجردة (AST) الخاصة بنا.

ثالثاً: بناء شجرة النحو المجردة (AST - Abstract Syntax Tree)

بعد حصولنا على شجرة الإعراب (Parse Tree) المبدئية من ANTLR، كان لابد من تحويلها إلى شجرة نحو مجردة (AST) خاصة بنا، حيث نتخلص من الرموز غير المهمة دلاليّاً (مثل الأقواس

والفواصل) ونحتفظ بالمعلومات المفيدة فقط. قمنا ببناء هيكل كائني (OOP) كامل لتمثيل العقد (Nodes) وتقسيمها إلى: 1. **عقد بايثون (Python Nodes)**: مثل `FunctionDefNode`, 2. `IfStatementNode`, `VariableDeclNode`. **عقد جينجا (Jinja Nodes)**: مثل `JinjaBlockNode`, `JinjaExpressionNode`, `HtmlElementNode`.

لتحقيق ذلك، استخدمنا **نمط الزائر (Visitor Pattern)**:

- **في بايثون**: قمنا بتعريف `PythonASTBuilderVisitor` الذي يمتد من زوار ANTLR الأساسيين لزيارة العقد وتحويلها إلى كائنات الـ AST التي برمجناها.
- **في جينجا**: بنينا زواراً مخصصين مثل `TemplateASTBuilder` لبناء شجرة تدمج بذكاء بين عناصر HTML وتعبيرات جينجا المعقدة.

تصدير الشجرة بصيغة (AST Serialization) JSON

من أهم متطلبات المشروع أن تتمكن من إخراج شجرة AST بصيغة مقروءة لتسهيل فحص الأخطاء وعمليات التتبع (Debugging). وهنا واجهتنا مشكلة (الدوران اللانهائي - Infinite Recursion) بسبب وجود مؤشرات عكسية في العقد (مثل `parent` و `children`).

لحل هذه المشكلة، برمجنا المُسلسل `ASTJsonSerializer.java` بالاعتماد على **مفهوم الانعكاس (Reflection)** في لغة جافا، بحيث:

- يمر الكود على كافة حقول الكلاس (Fields) في وقت التشغيل (Runtime).
- يستثني برمجياً الحقول التي تسبب تكراراً لانهائياً:

```
if(field.getName().equals("parent")) ||  
    field.getName().equals("scope")) continue
```
- يعالج الخرائط (Maps) والقوائم (Lists) بذكاء ليولد في النهاية ملفي `ast_python.json` و `ast_jinja.json`

مثال مصغر عن هيكل JSON الناتج لشجرة بايثون:

```
}  
  , "type": "ProgramNode_  
    ] : "statements"  
  }  
  , "type": "FunctionDefNode_  
    , "functionName": "index"  
    { ... } : "body"  
  {  
    [  
    {
```

رابعاً: بناء جدول الرموز (Symbol Table)

في هذه المرحلة، لم يعد تحليل "شكل" الكود كافياً، بل يجب علينا تتبع وفهم المتغيرات، الدوال، الكتل (Blocks)، والمجالات (Scopes). لذلك صممنا نظام مجالات هرمي (Scope Hierarchy):

1. **المجال العام (Global Scope):** يحتوي على المتغيرات والدوال المعرفة على مستوى الملف ككل.
2. **مجال الدوال والكتل (Function/Block Scope):** خاص بالمتغيرات المحلية (Local Variables).

1. جدول رموز بايثون (Python Symbol Table)

من خلال الكلاس `PythonSymbolTableBuilder`، نقوم بالمرور مجدداً على عقد الـ `AST`. عند العثور على تعريف دالة، نقوم بفتح مجال جديد `pushScope()`. وعند الخروج منها نغلقه `popScope()`. يتم تسجيل المتغيرات وتخزين معلوماتها مثل (اسمها، نوعها المتوقع، السطر البرمجي) لتسهيل إيجاد الأخطاء في المراحل القادمة.

2. جدول رموز جينجا (Jinja Symbol Table)

تحليل رموز جينجا كان أكثر تعقيداً لأن القوالب تحتاج للوصول للمتغيرات المُمَررة من بايثون، كما أنها تملك كتلتها البرمجية الخاصة (مثل حلقة `for`). من خلال `JinjaAstSymbolTableBuilder` قمنا بـ:

- تسجيل المتغيرات المُمَررة من تابع `render_template`.
- تسجيل متغيرات حلقة التكرار `for` بحيث لا يمكن الوصول إليها خارج الحلقة.
- تسجيل الكتل البرمجية `{% block %}` والفلاتر (Filters).

مثال برمجي يوضح كيف بنينا آلية فتح وإغلاق المجالات:

```
// عند الدخول في دالة أو كتلة جديدة في الـ AST
Scope functionScope = new Scope("Function_" + node.getFunctionName(),
                                ;currentScope)
;currentScope.addChild(functionScope)
;currentScope = functionScope // جعل المجال الحالي هو مجال الدالة

// بعد الانتهاء من زيارة العقد داخل الدالة (الخروج منها)
;currentScope = currentScope.getParent() // العودة للمجال الأب
```

خامساً: التحليل الدلالي ومعالجة الأخطاء (Semantic Analysis)

تعتبر هذه المرحلة من أهم مراحل المترجم، حيث يعمل المحلل الدلالي على فحص "المعنى المنطقي" للكود وليس شكله فقط (الذي اجتازه بنجاح في الـ `Parser`). ينقسم التحليل الدلالي في مشروعنا إلى قسمين: تحليل بايثون وتحليل جينجا2. قمنا ببناء الأصناف `PythonSemanticAnalyzer` و `JinjaAstSemanticAnalyzer` للقيام بهذه المهمة.

1. فحوصات لغة Python (تمت تغطية 22 فحص دلالي)

من خلال الاعتماد على `AST` و `ScopeStack`، قمنا بتنفيذ عدد كبير من الفحوصات تقسم إلى أخطاء (Errors) تمنع المترجم من الإكمال، وتحذيرات (Warnings).

أهم الأخطاء الدلالية (Errors):

1. استخدام تعليمة return خارج الدالة.
2. استخدام تعليمة break أو continue خارج حلقة التكرار (for/while).
3. تعريف الدالة أكثر من مرة بنفس الاسم (Duplicate function name).
4. تكرار اسم المعامل (Parameter) في نفس الدالة.
5. الإعلان عن متغير ك global بعد إسناد قيمة له محلياً.
6. القسمة على صفر بشكل صريح.
7. كتابة كود غير قابل للوصول بعد تعليمة return.
8. أخطاء الأنماط وعدم التوافق (Type Mismatch)، مثل جمع str و int.
9. دوران حلقة على كائن غير قابل للتكرار (Not iterable).
10. استخدام متغير خارج النطاق الخاص به (Out of scope) أو غير معرّف أساساً.

أهم التحذيرات (Warnings):

1. استيراد نفس المكتبة (Import) أكثر من مرة.
 2. استخدام اسم دالة كمتغير مما يؤدي إلى طمس الدالة الأصلية.
 3. تعريف متغير محلي دون استخدامه أبداً (Unused variable).
 4. استدعاء دالة لا يمكن الوصول إليها.
 5. تعريف دالة فارغة تحتوي فقط على pass.
 6. استخدام اسم قبل الإسناد إليه في النطاق المحلي.
 7. استدعاء دالة بعدد ضخم من المعاملات (> 7).
 8. المقارنة مع None و True/False باستخدام == بدلاً من is.
- مثال برمجي يوضح طريقة التقاط خطأ استخدام return خارج الدالة:

```
Override@
} public void visit(ReturnNode n)
// التحقق من العمق (functionDepth)
} if (functionDepth == 0)
error("استخدام تعليمة 'return' خارج الدالة", n.getColumn(), n.getLine());
{
    if (n.hasValue()) n.getValue().accept(this);
    return null;
}
```

2. فحوصات قوالب Jinja2 (تمت تغطية 17 فحص دلالي)

في قوالب جينجا، كان التحدي هو ربط المتغيرات ببيانات بايثون الوهمية (Mock Data).

أهم الأخطاء (Errors):

1. تعريف {% block %} بنفس الاسم أكثر من مرة في نفس الملف.
2. استخدام {% extends %} أكثر من مرة أو بشكل دائري (وراثة القالب لنفسه).
3. استدعاء ملف غير موجود في {% include %} أو {% import %} أو {% extends %}.

4. عدم تطابق التاغات (Tags) في HTML (مثلاً <p> يغلق بـ </div>).
5. استخدام متغير لم يتم تمريره من الدالة render_template في بايثون.
6. الوصول إلى خاصية (Attribute) غير موجودة في الـ Mock Data (مثل محاولة طباعة user.age بينما age غير متوفرة).
7. الوصول إلى فهرس (Index) خارج حدود القائمة الممررة (Index out of bounds).
8. استخدام الدالة url_for('static') للوصول لملف غير موجود مجلد static.

أهم التحذيرات (Warnings):

1. استخدام مرشح (Filter) غير معروف في جينجا.
2. استخدام متغير الحلقة loopVar بعد انتهاء الـ {% for %} الخاصة به.
3. إعادة تعريف متغير ممرر من بايثون باستخدام {% set %}.
4. كتابة كود جينجا قبل تعليمة {% extends %}.
5. عمل حلقة for على قائمة فارغة تماماً [].

مثال برمجي لكيفية فحص عدم تطابق تاغات HTML:

```

@Override
public void visit(HTMLNormalElementNode node)
{
    String opening = node.getTagName();
    String closing = node.getClosingTagName();
    if (closing != null && !opening.equals(closing))
    {
        error("عدم تطابق في التاغ (HTML): التاغ المفتوح <"+ opening + "> لا يطابق  

        التاغ المغلق >"+ closing + ">", node.getClosingLine(), + node.getClosingColumn());
    }
    return super.visit(node);
}

```

3. تقرير الأخطاء الدلالية (semantic_report.txt)

ينتهي عمل المحلل الدلالي بإنشاء ملف نصي شامل باللغة العربية يوضح أماكن الأخطاء والتحذيرات لتسهيل تتبعها من قبل المبرمج.

مثال لخرج حقيقي من التقرير المتولد:

```

=====
                                =====
                                تحليل الدلالات للغة بايثون (18 فحص)
=====
                                =====

```

التحذيرات (3):

Line 60:12 [WARNING] — المتغير 'product' معرّف ولكنه لم يُستخدم أبداً
 Line 84:16 [WARNING] — المتغير 'image_filename' معرّف ولكنه لم يُستخدم أبداً
 Line 110:12 [WARNING] — المتغير 'product_to_delete' معرّف ولكنه لم يُستخدم أبداً

الملخص: 0 خطأ، 3 تحذير = 3 إجمالي المشاكل

تحليل الدلالات لقوالب جينجا2: index.jinja (13 فحص)

لم يتم العثور على أي مشاكل دلالية.

الملخص: 0 خطأ، 0 تحذير = 0 إجمالي المشاكل

سادساً: توليد الكود وتصدير الصفحات (Code Generation)

بعد اجتياز الكود بنجاح لمرحلة التحليل الدلالي (Semantic Analysis) والتأكد من خلوه من الأخطاء، تنتقل إلى المرحلة ما قبل الأخيرة، وهي مرحلة **توليد الكود النهائي (Code Generation)**. حيث يقوم المترجم بدمج بيانات بايثون (Mock Data) مع قوالب جينجا لتوليد صفحات HTML نقية وثابتة (Static HTML) يمكن لأي متصفح قراءتها مباشرة.

1. محرك توليد الـ HTML

لقد قمنا بتصميم الكلاس `HtmlCodeGenerator` ليكون مسؤولاً عن هذه المهمة المعقدة. تتمثل مهامه الأساسية في:

- قراءة شجرة الـ AST الخاصة بـ Jinja2.
- ربط قيم المتغيرات القادمة من بايثون في الأماكن المخصصة لها في القالب (مثل `{{ product.name }}`).
- تنفيذ الحلقات `{% for %}` وتوليد عناصر HTML متكررة بناءً على طول القائمة.
- تنفيذ الشروط `{% if %}` وإخفاء أو إظهار كتل الـ HTML بناءً على النتيجة المنطقية.
- حل الوراثة `{% extends %}` والدمج `{% block %}` لدمج القالب الفرعي مع القالب الأساسي `base.jinja`.
- تحويل مسارات `url_for('static')` إلى مسارات نسبية صحيحة تشير لمجلد `static`.

مثال برمجي لطريقة تقييم حلقة `for` وتوليدها ككود HTML:

```
private String renderFor(ForBlockNode node)
;Object iterable = eval(node.getIterable())
;List<?> items = asList(iterable)

// إذا كانت القائمة فارغة وكان هناك {% else %} نقوم بتنفيذه
} if (items.isEmpty())
return node.hasElseBlock() ?
;"" : renderChildren(node.getElseBlock().getContent())
{

;()StringBuilder sb = new StringBuilder
```

```

;int i = 0
// الدوران على القائمة وتوليد الكود لكل عنصر
} for (Object item : items)
    pushScope // فتح مجال جديد لكل دورة
    // إسناد المتغير
    put(node.getVariable(), item)
    // توفير متغير 'loop' الخاص بجينجا
    put("loop", loopInfo(i, items.size()))
    // توليد المحتوى
    sb.append(renderChildren(node.getContent()))
    popScope // إغلاق المجال
    ++i
{
    return sb.toString
}

```

2. سجل التوليد (Generation Log)

مع كل عملية توليد يتم إنشاء ملف سجل يوثق الملفات التي تم إنتاجها والسياقات (Context) التي مُررت لكل ملف. مما يسهل عملية تتبع المخرجات. مثال لملف `generation_log.txt` المتولد:

```

HTML code generation log
=====
-----
Generated at 2026-08-30 22:54:07
Operation: compiler pipeline (Main) -- first create of HTML pages
Wrote ../output/index.html (context keys: [products])
Wrote ../output/product_detail.html (context keys: [product])
Wrote ../output/add_product.html (context keys: [])
Copied style.css

```

سابقاً: تشغيل الخادم الداخلي (OutputHttpServer) وربط كل شيء معاً

بما أننا نبني أداة تحاكي إطار عمل Flask بالكامل، فالتوليد لمرة واحدة لا يكفي! تخيل أن المستخدم قام بالضغط على زر "إضافة منتج"، كيف سيتم تحديث الصفحات؟

قمنا ببرمجة خادم HTTP خاص بنا مدمج في المشروع عبر الكلاس `OutputHttpServer`. يقوم هذا الخادم بالوظائف التالية:

1. **استضافة الصفحات (Hosting):** الاستماع على المنفذ (Port 8080) لتقديم صفحات ال HTML التي ولدها ال `HtmlCodeGenerator` للمتصفح.
2. **استقبال الطلبات (Routing):** استقبال طلبات مثل `GET /products` أو `POST ./add`

3. **التحديث الديناميكي للمحتوى (Dynamic Regeneration):** عندما يتلقى الخادم طلب POST لإضافة منتج أو حذفه، يقوم بـ:

- تحديث مصفوفة الـ products المحفوظة لديه في الذاكرة.
- استدعاء المحرك HtmlCodeGenerator ليقوم بتوليد جميع ملفات HTML من الصفر بالبيانات الجديدة.
- إعادة توجيه المستخدم لصفحة الـ Index ليرى المنتج الجديد وكأننا استخدمنا قاعدة بيانات حقيقية وإطار فلاسك الحقيقي!

مثال برمجي يوضح كيف يتعامل الخادم مع طلب POST لحذف منتج وإعادة توليد الصفحات:

```
private void handleDelete(HttpExchange ex, Integer id) throws
    IOException
    // 1. البحث عن المنتج وحذفه من المصفوفة
    products.removeIf(p -> idEquals(p.get("id"), id))

    // 2. إعادة توليد كامل صفحات الـ HTML بالبيانات المحدثة
    regenerate("DELETE product (id=" + id + ")", false)

    // 3. إعادة توجيه المتصفح للصفحة الرئيسية
    redirect(ex, "/")
}
```

الخاتمة

وبهذا نكون قد مررنا بجميع مراحل بناء هذا المترجم المتقدم، ابتداءً من قراءة النص المجرد (Lexical/Syntax) وبناء شجرة النحو المجردة (AST)، ومروراً ببناء جدول الرموز (Symbol Table) والفحوصات الدلالية الدقيقة (Semantic Analyzer)، وانتهاءً بتوليد الكود وعرضه بأسلوب ديناميكي عبر خادم HTTP مدمج. يُعد هذا المشروع محاكاة قوية وعميقة للمفاهيم التي تعمل خلف الكواليس في إطارات العمل الحديثة مثل Flask و Django.